

## INTERVIEW PREPARATION SERIES

# Java Control Flow

## Production War Stories

10 Scenario-Based Questions from Realistic Engineering Contexts

### ABOUT THIS DOCUMENT

**10 scenario-based questions** on Java control flow — each set in a realistic, messy, production engineering context. Every scenario tests whether you can trace through a bug, identify the language-specific cause, and apply the correct fix using Java's specific tools and idioms.

### SCENARIO 01

## The Guard That Crashed at Midnight

A financial reporting service runs a nightly batch job. It processes transactions, enriching each with metadata from a secondary database. Three weeks after launch, the job crashes at 00:17 every night with a null pointer exception in `enriched.getCategory()`. During the day, the same code runs without issue. The on-call engineer finds that transactions arriving after midnight have a different format — the secondary lookup returns null/None for them. The code reads:

```
if (txn.getAmount() > 0 && enriched.getCategory() != null
    && !enriched.getCategory().equals("EXCLUDED")) {
    process(txn);
}
```

**QUESTION**

**Why does this fail only after midnight? Trace the exact execution path that causes the crash. Fix the conditional and explain why condition ordering matters for correctness, not just style.**

**EXPECTED DEPTH**

- **Root cause: condition ordering violates the null guard.** After midnight, `enriched` is null/None. The expression evaluates left to right. `txn.amount > 0` is true (valid transaction), so evaluation proceeds to `enriched.getCategory()`. Dereferencing null crashes.
- **Why only after midnight:** daytime transactions all have enrichment data. Midnight-format transactions do not. The condition was tested on daytime data only.
- **Fix: check nullity first, apply short-circuit as a guard.**

```
if (txn.getAmount() > 0 && enriched != null
    && !"EXCLUDED".equals(enriched.getCategory())) {
```

- **The lesson:** null/None guards must *precede* any access on the null-able object. Reordering conditions to ‘clean them up’ without understanding dependency is dangerous. Document the required order with a comment when it’s non-obvious.

**SCENARIO 02**

## The Feature That Took a Week to Test

A pricing engine has a single `calculate_price()` function that has grown over 18 months to 180 lines with 8 levels of nested ifs. The QA team reports they need 5 days to regression-test a one-line change to the discount logic. The engineering lead asks you to estimate why QA is taking so long. You count the branching conditions: 23 if/elif/else branches total, 4 switch cases, 3 ternary operators. The function has never had a test coverage report run on it.

**QUESTION**

**Calculate the minimum number of test cases needed to achieve full branch coverage. Explain what cyclomatic complexity reveals about this function. Then propose a concrete refactoring plan that would reduce the testing burden, without rewriting the business logic.**

**EXPECTED DEPTH**

- **Cyclomatic complexity (CC) calculation:** Start at 1. Each `if/elif/else` branch adds 1. Switch: each `case` adds 1. Ternary: adds 1. Short-circuit operators (`&&` | `&or`) each add 1. Approximate CC from the description:  $1 + 23 + 4 + 3 = \sim 31$  + short-circuits.  $CC \approx 35-40$ . Minimum branch-coverage test cases = CC. That's 35-40 test cases for one function.
- **Why QA takes 5 days:** 40 paths × setup/teardown/validation time per test =  $\sim 5$  days. Each code change potentially invalidates multiple paths.
- **Refactoring plan (preserving logic):**
  - **Extract guard clauses:** move all validation ifs to the top as early returns. Reduces nesting by 2-3 levels immediately.
  - **Extract each discount tier to a named function:** `calculate_volume_discount()` `calculate_loyalty_discount()` etc. Each has  $CC \approx 3-5$  and needs 3-5 tests, not 40.
  - **Replace conditional dispatch with a strategy map:** map discount type to a function. Eliminates entire branches.
- **Target CC:**  $\leq 10$  per function. After refactoring, the orchestrator function should have  $CC \leq 5$ , with each extracted function having  $CC \leq 5$ . Total test cases:  $5 + 5 \times 4-7 = 40$  — same coverage, far more maintainable.

**SCENARIO 03**

## The Order That Disappeared

An e-commerce platform processes order cancellations nightly. The cancellation service maintains an in-memory list of active orders. When a cancellation is received, the service removes the order from the list and triggers a downstream event. A new engineer joins and writes the following during their first week:

```
for (Order order : activeOrders) {
    if (shouldCancel(order)) {
        activeOrders.remove(order);
        notifyCancelled(order);
    }
}
```

After deployment, customer support starts receiving reports of orders that were supposed to be cancelled still appearing in the system — and some orders that should still be active have been cancelled. The volume of incorrect cancellations is roughly 50% of what was expected — a suspicious ratio.

### QUESTION

**Why is exactly approximately 50% of the cancellations being processed correctly? Trace through the execution for a list [A, B, C, D] where B and D should be cancelled. Give the language-correct fix.**

### EXPECTED DEPTH

- **Root cause: mutation of the collection during iteration.**  
ConcurrentModificationException should fire — unless the collection is iterated by index, or the remove happens to prevent the exception check.
- **The 50% explanation (Python/Java list):** For [A, B, C, D] where B and D should be cancelled. Index 0: A — not cancelled. Index 1: B — cancelled, B removed, C shifts to index 1. Index 2: D — C was skipped entirely, D is processed but C was missed. Every other cancellation-candidate is skipped.
- **Correct fix:**

```
activeOrders.removeIf(order -> {
    if (shouldCancel(order)) {
        notifyCancelled(order);
        return true;
    }
    return false;
});
```

## SCENARIO 04

## The Loop That Never Stopped

A firmware engineer writes a backward-iterating loop to process sensor readings from newest to oldest:

```
long n = readings.size();
for (long i = n - 1; i >= 0; i--) {
    process(readings.get((int) i));
    if (i == 0) break; // "fix" added after first hang
}
```

The engineer added `if (i == 0) break` as a workaround after the loop hung. This 'fix' works but is a code smell hiding a logic issue. What's the real problem and clean solution?

## QUESTION

**Diagnose the exact cause of the bug (or misunderstanding). Explain the language-specific type behaviour. Give the idiomatic correct form.**

## EXPECTED DEPTH

- **Java has no unsigned types.** `long i` works correctly; it reaches `-1` and the condition `i >= 0` fails normally. The original loop would terminate correctly without the break.
- **The break was unnecessary** — it was added during debugging when something else was wrong (possibly `readings.get((int) i)` with a large `n` causing `int overflow`). The real problem was the cast.
- **Idiomatic Java backward iteration:**

```
for (int i = readings.size() - 1; i >= 0; i--)
    process(readings.get(i)); // size() fits int for all practical lists
```

## SCENARIO 05

## The Report That Took Three Hours

A SaaS analytics platform generates a weekly “cohorting” report: for each of 50,000 users, it finds which other users share the same subscription tier AND same signup month. In development with 200 test users, the report generates in 0.4 seconds. After launch with 50,000 users it takes 3 hours. The product team wants it to run in under 5 minutes. The core logic:

```
for (User user : users) {  
    for (User other : users) {  
        if (!user.id.equals(other.id)  
            && user.tier.equals(other.tier)  
            && user.signupMonth.equals(other.signupMonth)) {  
            cohorts.get(user.id).add(other.id);  
        }  
    }  
}
```

## QUESTION

**Calculate the exact operation count for  $n=200$  vs  $n=50,000$ . Explain why time scaled by that factor. Design an  $O(n)$  solution using grouping and give the time/space trade-off.**

## EXPECTED DEPTH

- **Exact counts:**  $n=200$ :  $200 \times 200 = 40,000$  comparisons.  $n=50,000$ :  $50,000 \times 50,000 = 2.5$  billion comparisons. Ratio:  $2.5 \times 10^9 / 4 \times 10^4 = 62,500\times$ . Time scaled from 0.4s to  $0.4 \times 62,500 = 25,000$  seconds  $\approx 7$  hours. Observed 3 hours — within the expected range with caching/branching optimisations.
- **$O(n)$  solution: group by composite key.**

```
// O(n) grouping
Map<String, List<String>> groups = new HashMap<>();
for (User u : users) {
    String key = u.tier + "_" + u.signupMonth;
    groups.computeIfAbsent(key, k -> new ArrayList<>()).add(u.id);
}
// Assign cohorts
for (List<String> group : groups.values())
    for (String uid : group)
        cohorts.put(uid, group);
```

- **Complexity:**  $O(n)$  for grouping +  $O(\text{total pairs})$  for output. If the average group size is  $k$ , output is  $O(n \times k)$ . Space:  $O(n)$  for the group map. For 50,000 users: milliseconds instead of hours.

## SCENARIO 06

# The Duplicate That Slipped Through

A payment deduplication service checks incoming payment requests against the last 1,000 processed payments to detect duplicates. The service lead is proud that it's never had a false negative. In a load test, duplicate payments start slipping through. The deduplication logic:

```
boolean isDuplicate = false;
outer:
for (Payment existing : recentPayments) {
    isDuplicate = true;
    for (String field : checkFields) {
        if (!payment.get(field).equals(existing.get(field))) {
            isDuplicate = false;
            continue outer;
        }
    }
    break outer;
}
```

**QUESTION**

Trace through an execution where the duplicate slips through. Identify the flag management bug. Rewrite the logic correctly.

**EXPECTED DEPTH**

- **The Java version using labeled break/continue is actually correct** — `continue outer` skips to the next existing payment when any field differs, `break outer` exits when all fields match. The `isDuplicate` flag reflects the last comparison correctly.
- **A common bug variant:** if `isDuplicate = true` is inside the inner loop instead of outside it, the same field-overwrite problem occurs. Always initialize the hypothesis *before* the inner loop.

**SCENARIO 07**

## The Search That Got Slower Every Day

A content platform's tag-search feature checks if an article contains any of the user's blocked tags. At launch with 500 articles, search returns in 3ms. Six months later with 80,000 articles, it takes 4 seconds. The PM says "just add more servers." The lead engineer says the problem is algorithmic. The search code:

```
outer:
for (Article article : articles) {
    for (String blocked : user.getBlockedTags()) {
        if (article.getTags().contains(blocked)) {
            exclude.add(article.getId());
            continue outer;
        }
    }
}
```



**QUESTION**

Give the true time complexity in terms of all three variables ( $n$ =articles,  $b$ =blocked tags per user,  $t$ =tags per article). Calculate the operation count for  $n=500$  vs  $n=80,000$  assuming  $b=20$ ,  $t=10$ . Then give the  $O(n \times b)$  solution.

**EXPECTED DEPTH**

- **True complexity:**  $O(n \times b \times t)$  where  $t$  comes from the inner membership test (`article.getTags().contains(blocked)` is  $O(t)$  for list search).
- **Operation count:**  $n=500$ ,  $b=20$ ,  $t=10$ : 100,000 ops.  $n=80,000$ : 16,000,000 ops. Ratio: 160 $\times$ . Time: 3ms  $\times$  160 = 480ms (should be). Actual 4s suggests additional overhead (cache misses, GC, string comparison).
- **Fix: convert article.tags to a Set before the loop —  $O(t)$  once,  $O(1)$  per lookup.**

```
Set<String> blocked = new HashSet<>(user.getBlockedTags()); // O(b)
for (Article article : articles) {                          // O(n)
    if (!Collections.disjoint(blocked, article.getTags())) { // O(t)
        exclude.add(article.getId());
    }
}
```

- **New complexity:**  $O(n \times t)$  for tag-set membership, plus  $O(b)$  pre-build. Scales linearly with articles.

**SCENARIO 08**

## The Retry Loop That Gave Up Too Early

A microservice calls an external payment gateway with automatic retry on transient failures. The retry logic was written by an engineer who's new to the codebase:

```
int attempts = 0;
Response response = new Response(); // default: success = false
while (attempts < MAX_RETRIES && !response.isSuccess()) {
    response = gateway.charge(payload);
    attempts++;
}
```

The service team notices payments succeed on the first attempt but the logs show the gateway being called twice. On investigation: the loop always makes two calls even when the first succeeds.

### QUESTION

**Trace through why the gateway is called twice even when the first attempt succeeds. Fix the loop using the correct loop construct for this problem.**

### EXPECTED DEPTH

- **Bug in Java:** `new Response()` may default to `success = false` but this is an artificial initial state. The while condition is evaluated before any gateway call. The second call: after success, `response.isSuccess()` is true. Condition: `1 < MAX_RETRIES && !true = false`. Exits correctly after one real call. But the log shows two calls — the logger was inside the while condition setup code. The real issue: the engineer added a diagnostic log that also triggers the call.
- **Clean do-while approach:**

```
int attempts = 0;
Response response;
do {
    response = gateway.charge(payload);
    attempts++;
} while (!response.isSuccess() && attempts < MAX_RETRIES);
```

### SCENARIO 09

## The Status Handler Nobody Wanted to Touch

A logistics platform routes parcels through 12 different status states (CREATED, PICKED, PACKED, LABELLED, DISPATCHED, IN\_TRANSIT, OUT\_FOR\_DELIVERY, DELIVERED, FAILED\_DELIVERY, RETURNED, LOST, CANCELLED). The status handler is a 200-line chain of `if/else if` statements. The team has been deferring adding two new states (HELD\_AT\_CUSTOMS, AWAITING\_PAYMENT) for three months because “the handler is too dangerous to touch.” Every status change in the last year has introduced at least one regression.

**QUESTION**

**Why is the if-else chain so fragile? Design the replacement using the appropriate language feature (switch expression, pattern matching, or dictionary dispatch). Show how adding a new status becomes a safe, isolated change rather than a risky global edit.**

**EXPECTED DEPTH**

- **Why the chain is fragile:** (1) Adding a new state requires finding the right insertion point in 200 lines — position matters if states share logic. (2) No compiler exhaustiveness check: a missing elif silently falls to the default. (3) Cyclomatic complexity  $\approx 14+$  — requires 14+ test cases for branch coverage. (4) Duplicate code: multiple states often do similar things with small variations, creating copy-paste bugs.
- **Java 21 fix: sealed interface + switch expression with exhaustiveness.**

```
sealed interface Status permits
    Created, Picked, InTransit, Delivered,
    FailedDelivery, HeldAtCustoms, AwaitingPayment { }

String handle(Status s) {
    return switch (s) {
        case Created c      -> onCreate(c);
        case InTransit t    -> onTransit(t);
        case Delivered d    -> onDelivered(d);
        case HeldAtCustoms h -> onHeld(h);
        case AwaitingPayment a -> onPayment(a);
        // Compiler enforces: all permitted types must be handled
        // Adding a new Status subclass = compile error until handled
    };
}
```

- **The sealed interface** makes the type hierarchy closed: only declared subclasses exist. The switch expression is exhaustive by the compiler. Adding a new status that isn't handled in the switch is a compile-time error — regressions become impossible to ship.

## SCENARIO 10

## The Matrix Report That Grew Quadratically

A data science team builds a similarity matrix for 5,000 product SKUs. For every pair of SKUs, they compute a similarity score. The engineer writes a nested loop. In testing with 100 SKUs it runs in 0.3 seconds. With 5,000 SKUs it never finishes — the team cancels after 40 minutes. The head of data says “just write it in a lower-level language.” The lead engineer says the problem isn’t the language.

## QUESTION

**Calculate the exact operation counts for  $n=100$  vs  $n=5,000$ . Explain why switching languages doesn’t help. Then describe three algorithmic improvements that reduce the effective work: (1) eliminating redundant symmetric pairs, (2) early exit from similarity computation, and (3) approximate nearest-neighbour techniques.**

## EXPECTED DEPTH

- **Exact counts:** full matrix:  $n=100$ : 10,000 pairs.  $n=5,000$ : 25,000,000 pairs. Ratio:  $2,500\times$ . Time:  $0.3s \times 2,500 = 750$  seconds  $\approx 12.5$  minutes. With 40+ minutes observed, each pair’s similarity computation is  $O(m)$  where  $m$  = feature vector length — the true complexity is  $O(n^2 \times m)$ .
- **Why language change doesn’t help:** a  $10\times$  language speedup reduces 12.5 minutes to 75 seconds. That’s better, but still  $O(n^2)$ . At  $n=50,000$  SKUs, it’s back to the original problem. Algorithmic change is needed, not implementation change.
- **Improvement 1: symmetric elimination.**  $\text{similarity}(A, B) == \text{similarity}(B, A)$ . Only compute the upper triangle:  $n(n-1)/2$  pairs instead of  $n^2$ . For  $n=5,000$ : 12.5 million pairs instead of 25 million.  $2\times$  speedup. `for j = i+1`.
- **Improvement 2: early exit.** Set a minimum similarity threshold. During similarity computation (e.g., dot product), if the running sum cannot possibly reach the threshold given remaining terms, stop early.  $O(m)$  best case drops to  $O(k)$  where  $k < m$ .
- **Improvement 3: approximate nearest neighbour (ANN).** Algorithms like LSH (Locality-Sensitive Hashing), FAISS, or Annoy index all SKUs in  $O(n \times m)$  preprocessing, then find approximate nearest neighbours in  $O(m \log n)$  per query instead of  $O(n \times m)$ . Error rate is controllable. Standard approach in production recommendation systems.

- 
- **Combined:** ANN reduces the problem from  $O(n^2 \times m)$  to  $O(n \times m \times \log n)$ . For  $n=5,000$ ,  $m=200$ : from 5 billion to 11 million operations. Runs in seconds.